

Hai Yen's Notes

Table of Contents

Smoke Test for the two cameras (Regular vs NoIR).....	2
Experimental method:.....	2
Procedure per smoke level:.....	2
Experiment Table.....	2
Dark Environment.....	2
Light Environment.....	3
Overall Results.....	4
Potential switch to USB Cameras.....	4
Initial Object Detection.....	4
Setup.....	5
Determining the YOLO version to use.....	5
Testing different YOLO models.....	6
Object Detection in a Low-Light environment.....	8
Testing different YOLO models in Low Light.....	8
YOLO Object Detection Pipeline Development.....	10
Initial Approach & Limitations.....	10
Revised System Architecture.....	10
Final Pipeline Design.....	10
Server-Side YOLO Setup.....	11
API-Based YOLO Detection (Initial Version).....	11
Workflow.....	11
Transition to Background Watcher System.....	12
Problem with API Approach.....	12
Solution: YOLO Watcher Service.....	12
YOLO Watcher Pipeline.....	12
Workflow.....	12
File Handling Improvements.....	13
Filtering for Living Objects (Key Feature).....	13
Goal.....	13
Living Object Definition.....	14
Efficient Filtering Design.....	14
Problem.....	14
Final Solution: Metadata-Based Filtering.....	14
Frontend Behavior.....	15
Performance Observations.....	15
Final YOLO System Behavior Overview.....	16

Smoke Test for the two cameras (Regular vs NoIR)

Experimental method:

Will have different smoke levels and both cameras will capture images before increasing the smoke level (using incense sticks)

Procedure per smoke level:

1. Set the smoke level using incense sticks
2. Wait 60 seconds for smoke to stabilize
3. Capture images with first Camera (Regular)
4. Switch cameras with multiplexer
5. Wait ~5 sec for camera to stabilize
6. Capture images with second camera (NoIR)

Photo File Naming Convention:

Smokelevel_camera.jpg

Experiment Table

Explanations:

Contrast CDI measures how well the camera can distinguish between the bright white stripes and dark black stripes on the target pattern, so the higher the number, the better.

It ranges from 0 to 1:

- 1.0 = perfect: the camera sees pure white and pure black, no smoke interference
- 0.0 = completely degraded: everything looks the same gray, the stripes are invisible

Dark Environment

Lighting	Smoke Density (sticks)	Camera	Contrast (CDI)	Sharpness	Edges
dark	0	noir	0.5556	2.46	0
dark	0	regular	0.6667	2.36	0
dark	1	noir	0.7429	2.64	461
dark	1	regular	0.7674	2.41	0

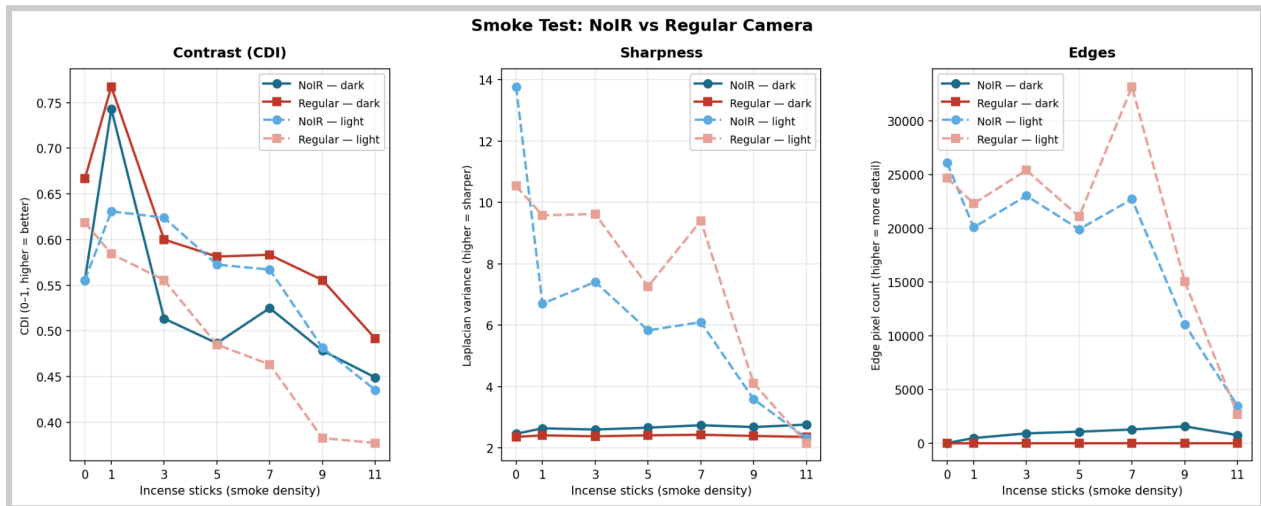
Hai Yen's Notes

dark	3	noir	0.5135	2.6	899
dark	3	regular	0.6	2.38	0
dark	5	noir	0.4865	2.66	1055
dark	5	regular	0.5814	2.41	0
dark	7	noir	0.525	2.74	1261
dark	7	regular	0.5833	2.43	0
dark	9	noir	0.4783	2.68	1553
dark	9	regular	0.5556	2.39	0
dark	11	noir	0.449	2.76	737
dark	11	regular	0.4915	2.36	0

Light Environment

Lighting	Smoke Density (sticks)	Camera	Contrast (CDI)	Sharpness	Edges
light	0	noir	0.5556	13.77	26087
light	0	regular	0.6186	10.53	24691
light	1	noir	0.6309	6.7	20078
light	1	regular	0.5844	9.58	22290
light	3	noir	0.6239	7.41	23033
light	3	regular	0.5556	9.62	25411
light	5	noir	0.5726	5.83	19906
light	5	regular	0.485	7.26	21092
light	7	noir	0.5671	6.1	22715
light	7	regular	0.4632	9.4	33191
light	9	noir	0.4815	3.59	11033
light	9	regular	0.3826	4.11	15041
light	11	noir	0.4352	2.32	3466
light	11	regular	0.3772	2.16	2679

Overall Results



- Smoke consistently degrades image quality across all three metrics
- In darkness, the NoIR camera significantly outperforms the regular camera (the regular camera detects almost no edges at any smoke level while the NoIR continues to detect edges throughout)
- In normal lighting, both cameras perform similarly

Potential switch to USB Cameras

- 640x480 Output which is a lower resolution than what we wanted, but this should still be good enough to do the expected tasks
 - Lower resolution -> less bandwidth
 - Main concern is the Pi being able to handle all 3 camera streams while maintaining stable cameras
- ❖ Made the final decision to keep the NoIR Picamera cameras because of budget, waste of already purchased materials as well as performance concerns.

Initial Object Detection

Previously, I tried running the YOLO object detection model on the raspberry pi, however it would take a lot of computing power, so the new workflow will be to send the

Hai Yen's Notes

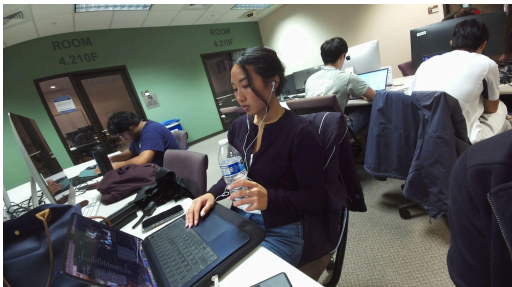
captured images on all three pi's to a server, and perform the object detection on the server or VM.

Setup

- Took 5 test images on the raspberry pi
- Downloaded the images to a subfolder 'yolo_test_images' on my Mac that is within a folder 'yolo'
- Cd into the yolo folder
- Created a venv for the yolo work
 - `python3 -m venv yolo_env`
 - `source yolo_env/bin/activate`

Sample Images taken

Image1.jpg	Image2.jpg	Image3.jpg
		

Image4.jpg	Image5.jpg
	

Determining the YOLO version to use

The most lightweight and compatible version for the needed tasks of our project is the YOLO11 model.

Hai Yen's Notes

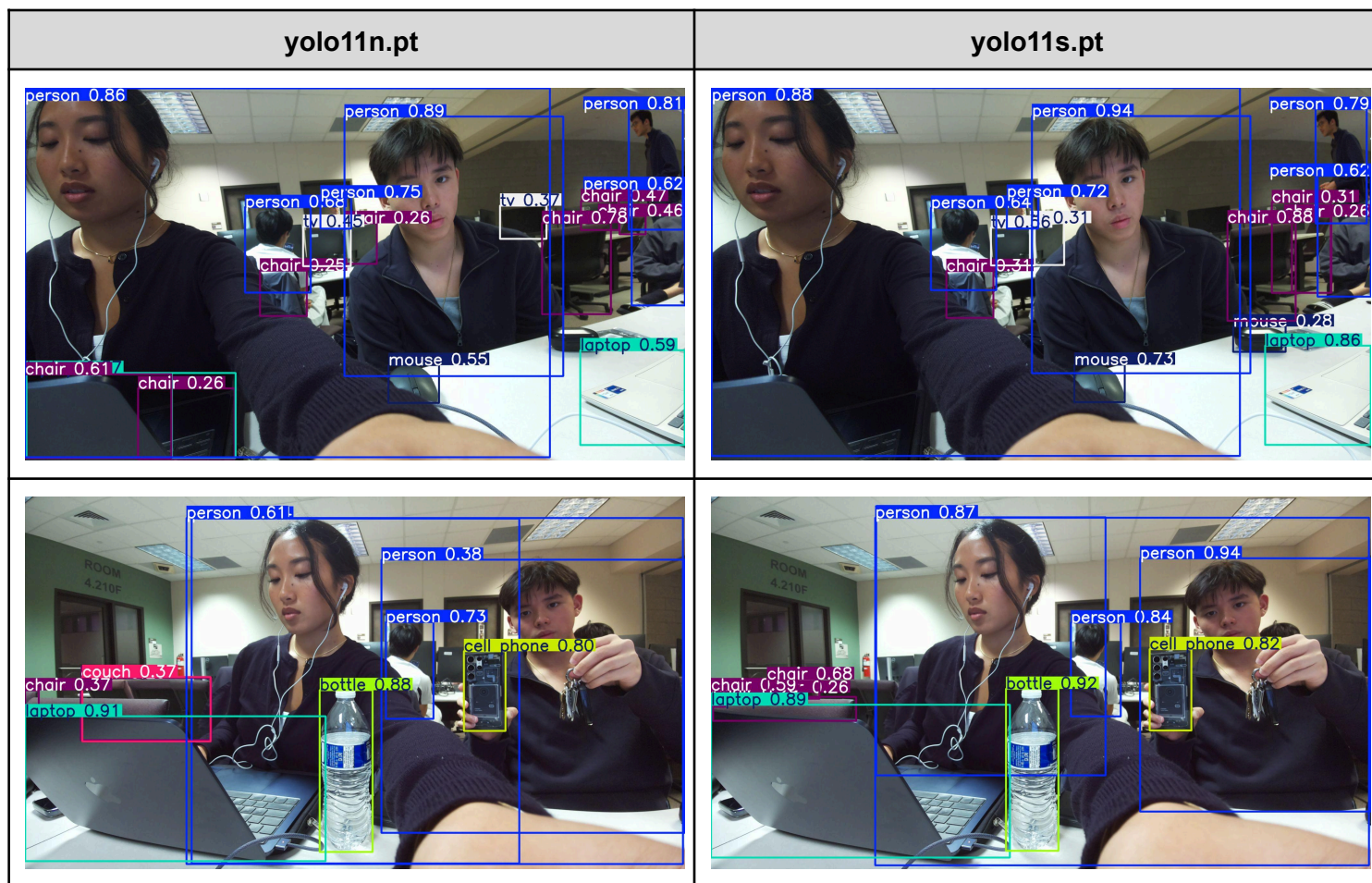
The YOLO11 architecture is able to detect, segment, and classify a wide range of objects in real-time, including standard COCO dataset classes (people, vehicles, animals, household items)

- This is suitable for our project, where identifying living objects is crucial

Testing different YOLO models

The first model tested was the [yolo11n.pt](#) (YOLO11 nano) model. I researched another model, the [yolo11s.pt](#) (YOLO11 small) model, which is a bit slower than the nano model, but it is more accurate in low-light conditions.

- **Nano Test Stats:**
Speed: 2.7ms preprocess, 31.5ms inference, 1.3ms postprocess per image at shape (1, 3, 384, 640)
- **Small Test Stats:**
Speed: 2.6ms preprocess, 84.5ms inference, 1.5ms postprocess per image at shape (1, 3, 384, 640)



Hai Yen's Notes



Next steps and Tasks to do

- Run YOLO on low-light environment images
- Make sure it works with the server architecture we have
- Determine the pipeline from object detection pictures -> 3D map from gaussian splat

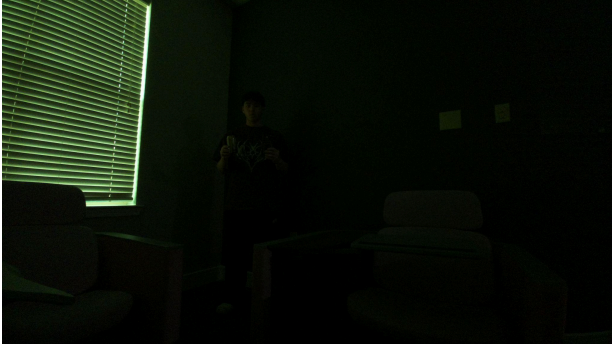
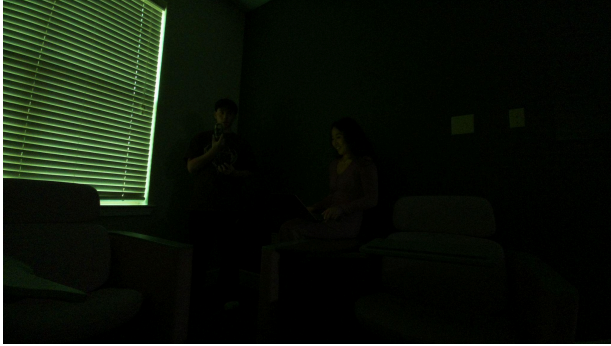
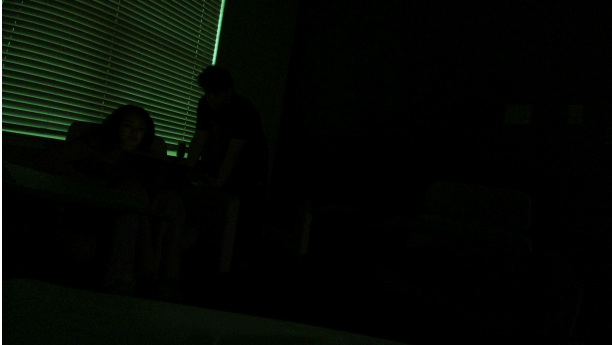
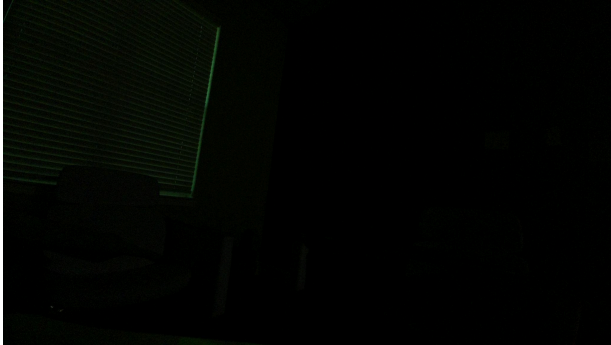
Hai Yen's Notes

Useful Links

- <https://docs.ultralytics.com/datasets/detect/coco/#dataset-yaml>
 - Lists the COCO dataset that shows which object corresponds to which number
 - Can be used to then create custom alerts/pings based on whether a living object was detected in the image
-

Object Detection in a Low-Light environment

Sample Images taken

Image1.jpg	Image2.jpg
	
Image3.jpg	Image4.jpg (no living objects)
	

Testing different YOLO models in Low Light

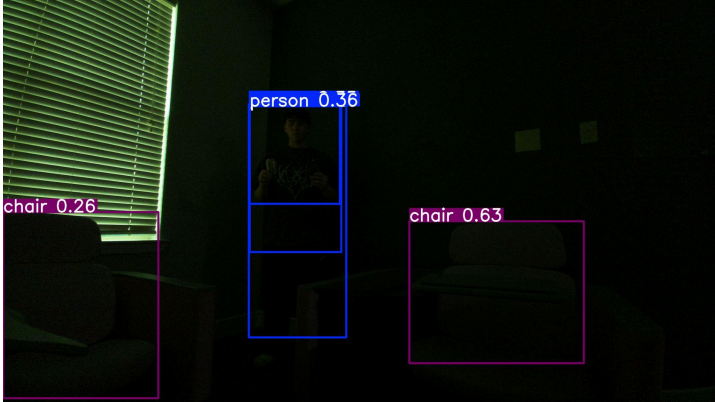
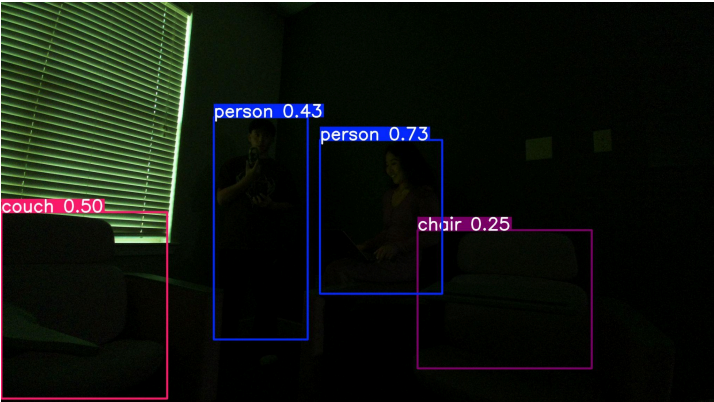
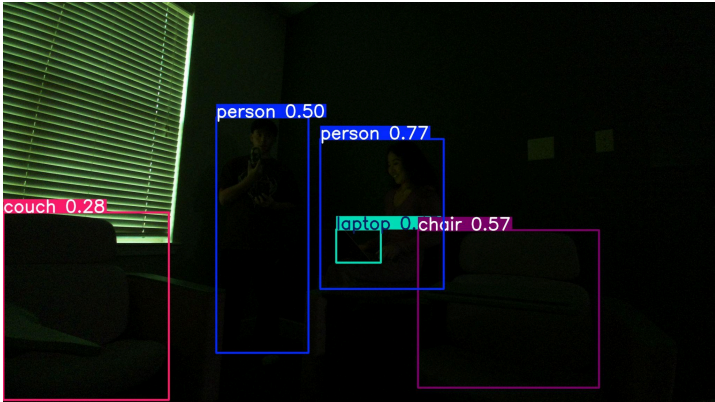
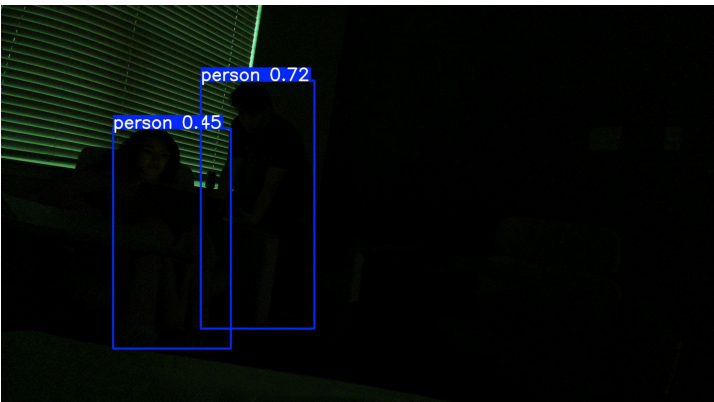
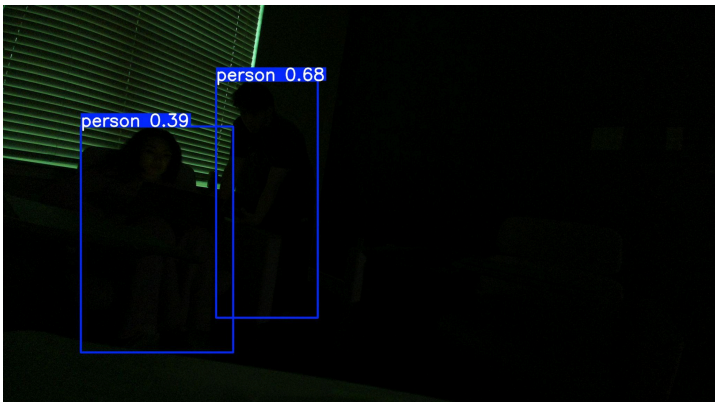
- Nano Test Stats:

Hai Yen's Notes

Speed: 2.4ms preprocess, 31.8ms inference, 1.8ms postprocess per image at shape (1, 3, 384, 640)

- **Small Test Stats:**

Speed: 2.5ms preprocess, 59.9ms inference, 1.9ms postprocess per image at shape (1, 3, 384, 640)

yolo11n.pt	yolo11s.pt
 Detection results for yolo11n.pt (Image 1): - person: 0.73 - couch: 0.51 - chair: 0.34	 Detection results for yolo11s.pt (Image 1): - person: 0.36 - chair: 0.26 - chair: 0.63
 Detection results for yolo11n.pt (Image 2): - person: 0.43 - person: 0.73 - couch: 0.50 - chair: 0.25	 Detection results for yolo11s.pt (Image 2): - person: 0.50 - person: 0.77 - couch: 0.28 - laptop: 0.0 - chair: 0.57
 Detection results for yolo11n.pt (Image 3): - person: 0.72 - person: 0.45	 Detection results for yolo11s.pt (Image 3): - person: 0.68 - person: 0.39



- ❖ Since the Test Stats and processing times for both models are nearly identical, I will use the [YOLO11s.pt](#) (small version) since it picks up on detections better in the low-light environment with more confidence

YOLO Object Detection Pipeline Development

Initial Approach & Limitations

Initially, object detection using YOLO was attempted directly on the Raspberry Pi. However, this approach was not feasible due to:

- High computational demand of YOLO models
- Limited CPU/GPU resources on the Raspberry Pi
- Increased latency and unstable performance when running alongside camera capture

Conclusion:

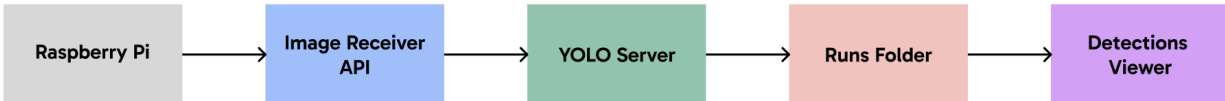
Object detection must be offloaded to a more powerful machine (Mac or remote server).

Revised System Architecture

Final Pipeline Design

1. Raspberry Pi captures images (multiple cameras)
2. Images are sent to a server via HTTP
3. Server performs YOLO inference
4. Annotated images are saved
5. Detection results are displayed in a frontend viewer

Hai Yen's Notes



Server-Side YOLO Setup

Environment Setup

```
python3 -m venv yolo_env
source yolo_env/bin/activate
pip install ultralytics fastapi uvicorn pillow numpy torch
```

Model Selection

Tested:

- yolo11n.pt (nano)
- yolo11s.pt (small)

Tradeoff:

- **Nano:** faster, less accurate
- **Small:** slower, better detection (especially low-light)

Decision:

Use `yolo11s.pt` for better reliability in real-world conditions

API-Based YOLO Detection (Initial Version)

Endpoint

```
POST /detect
```

Workflow

1. Receive image via upload
2. Convert to numpy array
3. Run YOLO inference
4. Extract detections (class, confidence, bbox)
5. Save:
 - Raw image
 - Annotated image

6. Return detection metadata

Example Output

```
0: 384x640 1 person, 1 laptop, 21.9ms  
Speed: 1.7ms preprocess, 21.9ms inference, 14.0ms postprocess
```

Transition to Background Watcher System

Problem with API Approach

- Required manual triggering (/detect)
- Not scalable for continuous image streams
- Did not integrate cleanly with multi-camera pipeline

Solution: YOLO Watcher Service

A background service was created to:

- Continuously monitor a directory (incoming_images_png)
 - Automatically process new images
 - Remove need for manual API calls
- ❖ This change also allows for both YOLO and SLAM to use the same images since they are taking images from the same incoming stream so that there are no mismatches
-

YOLO Watcher Pipeline

Workflow

1. Image Receiver saves incoming images to:

```
incoming_images_png/
```

2. YOLO watcher:
 - Detects new files
 - Waits until file is fully written
 - Runs YOLO inference
 - Saves annotated images to:

```
runs/api_outputs/annotated/
```

3. Frontend polls:

```
GET /images
```

File Handling Improvements

Issues Encountered

- YOLO processing partially written images
- File access conflicts
- Missing directories after deletion

Fixes

- Added file age check:

```
MIN_FILE_AGE_SEC = 1.0
```

- Added directory auto-creation:

```
mkdir(parents=True, exist_ok=True)
```

- Added safe file handling using:

```
ImageFile.LOAD_TRUNCATED_IMAGES = True
```

Filtering for Living Objects (Key Feature)

Goal

Only show images with **living objects** in the detection UI while still saving **all annotated images**

Living Object Definition

Based on COCO dataset:

```
LIVING_CLASSES = {"person", "cat", "dog", "bird", "horse", "sheep",  
"cow"}
```

Efficient Filtering Design

Problem

Re-running YOLO during `/images` was inefficient:

- Doubles computation
- Slows down UI
- Not scalable

Final Solution: Metadata-Based Filtering

Instead of re-running YOLO:

Step 1: During Detection

After inference, determine if image contains a living object:

```
has_living = False  
for box in r.bboxes:  
    cls_name = model.names[int(box.cls[0])]   
    if cls_name in LIVING_CLASSES:  
        has_living = True  
        break
```

Step 2: Save Metadata File

For each annotated image:

```
annotated_XXXX.jpg  
annotated_XXXX.txt
```

Metadata file contents:

```
living=1 (if detected)
living=0 (if not)
```

Step 3: Filter in **/images** Endpoint

```
meta = ANN_DIR / (f.stem + ".txt")
if meta.exists() and meta.read_text().strip() == "living=1":
    include image
```

Frontend Behavior

Detection Viewer

- Polls /images every 2 seconds
- Only receives filtered images
- Displays:
 - Annotated image
 - Timestamp (filename)
 - Notification popup
- ❖ No frontend changes required
 - Filtering is handled entirely in backend
- ❖ Made some frontend changes so that the images were displaying in a nice grid rather than being compressed
 - Made the page scrollable to not limit how many pictures can be displayed
 - Changed the flex grid that was causing problems in the body, making pictures look compressed

Performance Observations

YOLO11s Performance

Typical:

- ~20–60 ms inference per image

Pipeline Efficiency

- No redundant YOLO runs

Hai Yen's Notes

- Real-time processing achievable
 - Works with continuous image stream
-

Final YOLO System Behavior Overview

- All images are processed and saved
- Only relevant (living-object) images are displayed
- Pipeline runs automatically without manual triggering
- System supports real-time multi-camera input